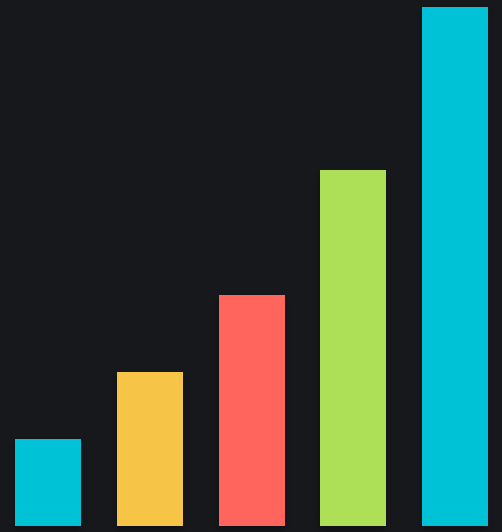
# My Dishonest Benchmark

How to Manufacture a 100x PostgreSQL Performance Claim



**Mayur@Veeam**

AGENDA

# This is not a “look at my TPS” talk.

The goal: recognize where **benchmarking** ends and **benchmarketing** begins.



# INSPIRATION



**Average Database CEO** @AvgDatabaseCEO · Feb 24

New “spacetime” DB takes shots at @convex @supabase @PlanetScale @CockroachDB @tursodatabase

Compares Convex to using “paper and pencil.”



**max** @max_kelly · Feb 24

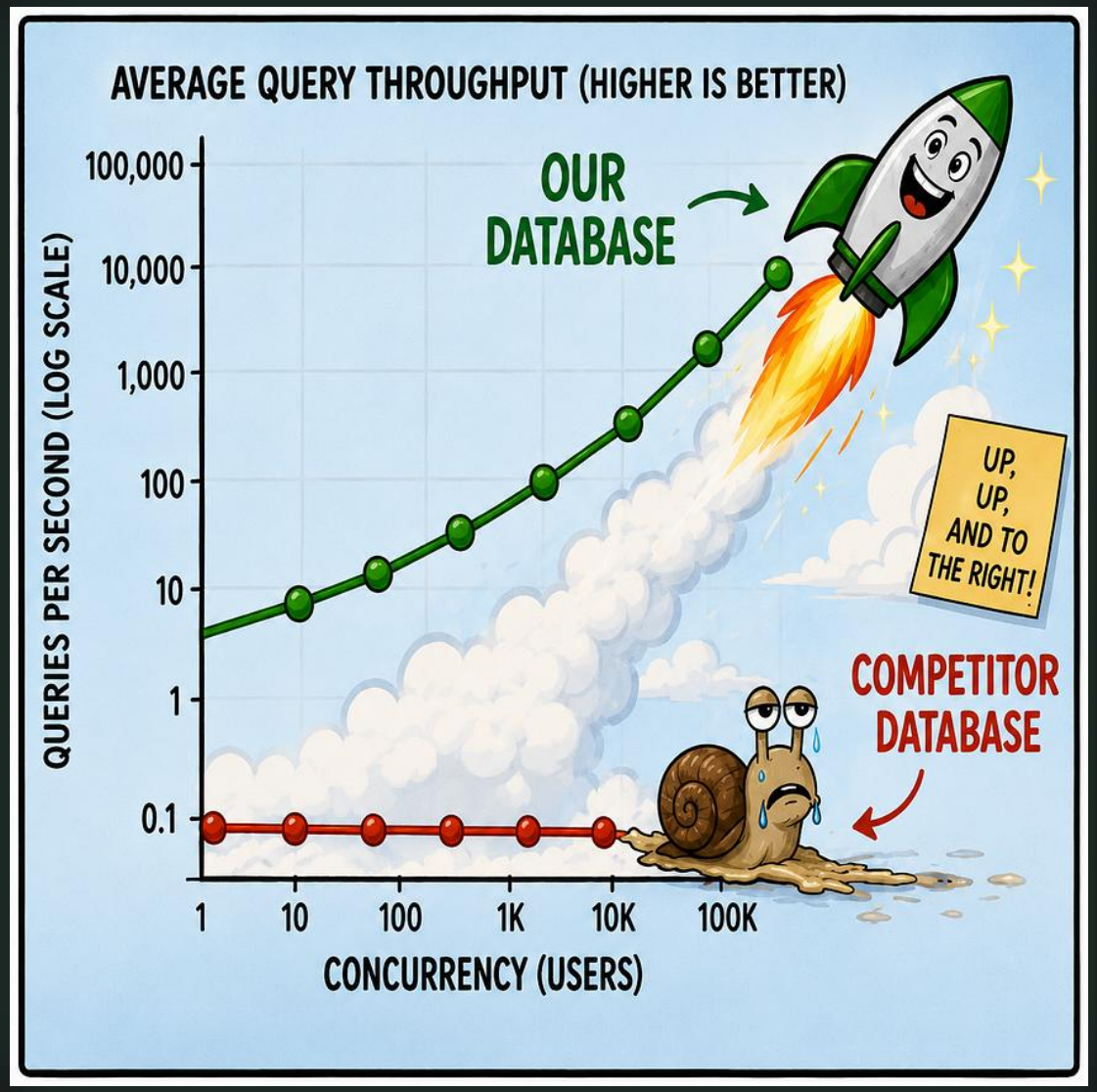
Replying to @RayFernando1337

This is lowkey savage...




MY DISHONEST BENCHMARK

# MODERN DAY BENCHMARKING



# HISTORY

## BENCHMARKING DATABASE SYSTEMS
## A SYSTEMATIC APPROACH

Dina Bitton
David J. DeWitt
Carolyn Turbyfill

Computer Sciences Department
University of Wisconsin-Madison

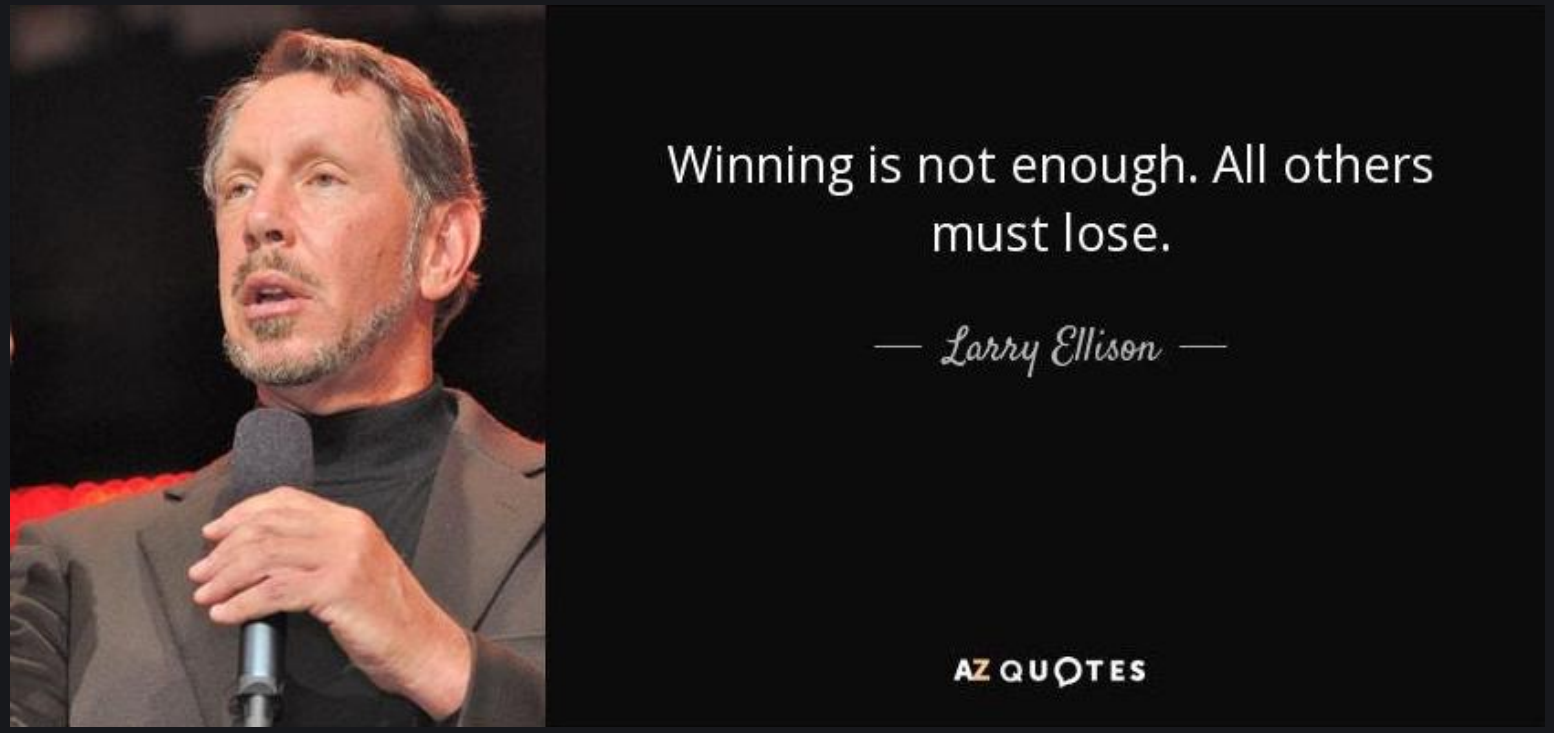
ABSTRACT

This paper describes a customized database and a comprehensive set of queries that can be used for systematic benchmarking of relational database systems. Designing this database and a set of carefully tuned benchmarks represents a first attempt in developing a scientific methodology for performance evaluation of database management systems. We have used this database to perform a comparative evaluation of the "university" and "commercial" versions of the INGRES database system, on one hand, and the IDM 500 database machine, on the other hand. We present a subset of our measurements (for the single user case only), that constitute a preliminary performance evaluation of these systems.

(DAC) [IDM500, EPST80, UBEL81] to the "commercial" and "university" versions of the INGRES database system [STON76, STON80].[2]

Since database machines have already been an active field of research for an entire decade, and a few machines have now been implemented, we feel that the time has come for measuring the actual performance enhancement that can be expected from using special purpose hardware and software for database management. When database machines designs such as CASSM [SU75], RAP [OZKA75, OZKA77], DBC [BANE78], DIRECT [DEWI79] were proposed, the future of database machines looked bright. It seemed that both successful research efforts and advances in hardware technology would lead to the widespread use of commercial database machines. However, while most projects appeared promising initially, it is only very recently

# There was one small problem.



In 1983, at the University of Wisconsin, Dina Bitton, David DeWitt, and Carolyn Turbyfill created a database benchmarking framework. included (lower is better):

Join without indices

| system | joinAselB | joinABprime | joinCselAselB |
|---|---|---|---|
| U-INGRES | 10.2 | 9.6 | 9.4 |
| C-INGRES | 1.8 | 2.6 | 2.1 |
| ORACLE | > 300 | > 300 | > 300 |
| IDMnodac | > 300 | > 300 | > 300 |
| IDMdac | > 300 | > 300 | > 300 |
| DIRECT | 10.2 | 9.5 | 5.6 |
| SQL/DS | 2.2 | 2.2 | 2.1 |

Join with indices, primary (clustered) index

| system | joinAselB | joinABprime | joinCselAselB |
|---|---|---|---|
| U-INGRES | 2.11 | 1.66 | 9.07 |
| C-INGRES | 0.9 | 1.71 | 1.07 |
| ORACLE | 7.94 | 7.22 | 13.78 |
| IDMnodac | 0.52 | 0.59 | 0.74 |
| IDMdac | 0.39 | 0.46 | 0.58 |
| DIRECT | 10.21 | 9.47 | 5.62 |

# THE DEWITT CLAUSE

## The DeWitt Clause changed the battlefield

1980S

### Wisconsin Benchmark

Independent academic DBMS benchmarking

VENDOR PRESSURE

### Commercial conflict

Poor results trigger fights

DEWITT CLAUSE

### License restriction

Publishing results can require approval

RESULT

### Benchmarketing

Loss of academic interest

CONSEQUENCE

# The DeWitt Clause helped change who controlled the benchmark narrative

INDEPENDENT RESEARCHERS

## Pulled back

Became less willing to publish comparative results

VENDORS

## Stepped in

Increasingly published their own benchmark results

# BENCHMARK ANARCHY

## Six ways to tilt a benchmark favorably towards your database

**01**

**Tune**

your own database carefully

**02**

**Default**

the comparison database

**03**

**Choose**

a workload your architecture loves

**04**

**Ignore**

cost + tail latency

**05**

**Ignore**

write amplification + crash behavior

**06**

**Publish**

the one graph where you win

TRANSPARENCY

# A better fight: publish everything



01

## CODE

Exact harness + implementation



02

## CONFIG

Settings on every system



03

## COST

Hardware + spend



04

## WORKLOAD

What each side actually did

# PostgreSQL has a marketing problem.

No owner. No marketing department. No PR strategy. No launch-day rocket-vs-snail charts.

# THE EXPERIMENT

## So let us create a heroic graph

**100,000+**

**OPERATIONS / SECOND**

*by hook or crook*

**CONSTRAINT**

Every reported number comes from a real run.

**FREEDOM**

I can quietly change what the number means.

REPRODUCTION PACK

# Dishonest does not mean unreproducible



01

schema + pgbench workloads



02

reset scripts + cheat variants



03

result collection



04

chart/report generation code



github.com/secp256k1-sha256/talks_and_blogs/tree/main/dishonest_benchmark

# Tiny TPC-C-flavoured accounting.

Not TPC-C. Not compliant. Just OLTP-shaped enough to feel familiar.

# SCHEMA.SQL

## Four tables, seeded with 100 branches and 1M accounts

### CORE TABLES

```sql
CREATE TABLE bench_branches (
  branch_id int PRIMARY KEY,
  balance bigint NOT NULL DEFAULT 0
);

CREATE TABLE bench_accounts (
  account_id bigint PRIMARY KEY,
  branch_id int NOT NULL
    REFERENCES bench_branches(branch_id),
  balance bigint NOT NULL DEFAULT 0
);
```

### LEDGER AND AUDIT TABLES

```sql
CREATE TABLE bench_ledger (
  ledger_id bigserial PRIMARY KEY,
  account_id bigint NOT NULL,
  branch_id int NOT NULL,
  amount int NOT NULL,
  created_at timestamptz NOT NULL
    DEFAULT clock_timestamp()
);

CREATE TABLE bench_audit (
  audit_id bigserial PRIMARY KEY,
  account_id bigint NOT NULL,
  action text NOT NULL,
  created_at timestamptz NOT NULL
    DEFAULT clock_timestamp()
);
```

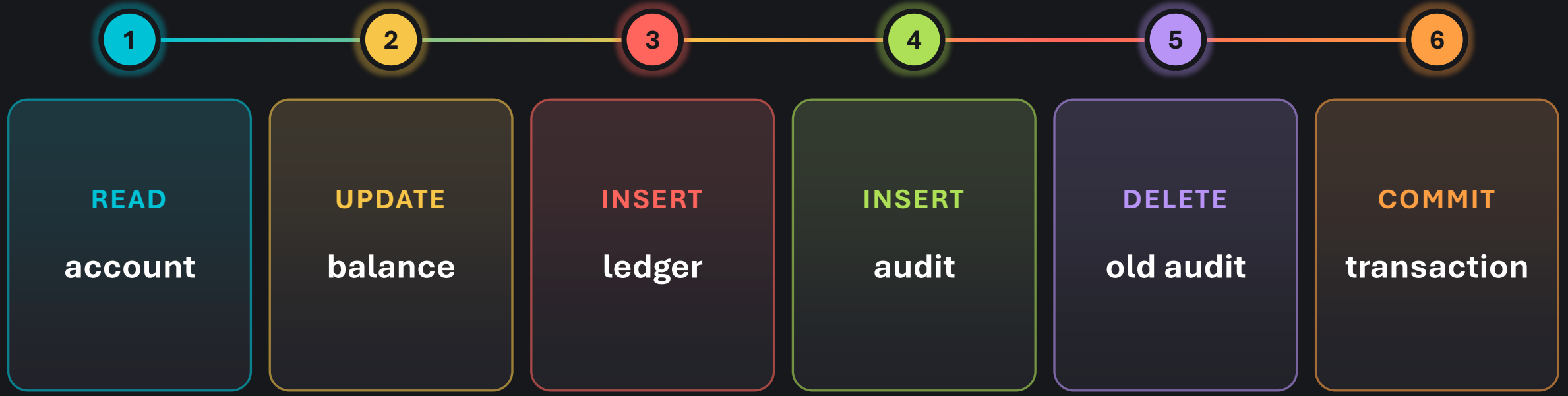
### SEED DATA

```sql
INSERT INTO bench_branches(branch_id)
SELECT g
FROM generate_series(1, 100) AS g;

INSERT INTO bench_accounts
  (account_id, branch_id, balance)
SELECT g, 1 + (g % 100), 100000
FROM generate_series(1, 1000000) AS g;
```

# THE WORKLOAD

## One transaction does six things



# PGBENCH

## 32 clients run this five-statement transaction for 120 seconds

### 1 SETUP, READ, UPDATE

```
-- pgbench script for tests
\set aid random(1, 1000000)
\set delta random(-500, 500)

BEGIN;

SELECT balance
FROM bench_accounts
WHERE account_id = :aid;

UPDATE bench_accounts
SET balance = balance + :delta
WHERE account_id = :aid;
```

### 2 LEDGER AND AUDIT WRITES

```
INSERT INTO bench_ledger
  (account_id, branch_id,
   amount)
SELECT account_id, branch_id,
       :delta
FROM bench_accounts
WHERE account_id = :aid;

INSERT INTO bench_audit
  (account_id, action)
VALUES (:aid, 'debit_credit');
```

### 3 CLEANUP AND COMMIT

```
DELETE FROM bench_audit
WHERE audit_id IN (
  SELECT audit_id
  FROM bench_audit
  WHERE created_at < now()
    - interval '10 minutes'
  ORDER BY created_at
  LIMIT 1
);

COMMIT;
```

# PGBENCH

## The run: 32 clients on 8 threads for 120 seconds, no vacuum

### RUN COMMAND

```
----*** Run pgbench ***----
pgbench -n -c 32 -j 8 -T 120 --protocol=simple \
  -f indexed_baseline.sql pgbench_darkside
```

### WHAT EACH FLAG DOES

| | | | |
|---|---|---|---|
| `-n` | Skip the pgbench pre-run vacuum | `--protocol=simple` | Simple query protocol |
| `-c 32` | 32 concurrent client sessions | `-f` | Run the custom script file |
| `-j 8` | 8 worker threads | `pgbench_darkside` | Target database |
| `-T 120` | Run for 120 seconds | | |

# TEST SETUP

**100**

BRANCHES

**1,000,000**

ACCOUNTS

**32**

CLIENTS

**8**

PGBENCH JOBS

**120 s**

PER RUN

**simple**

PROTOCOL

# Benchmark environment

One desktop machine, one local PostgreSQL instance, 120-second runs.

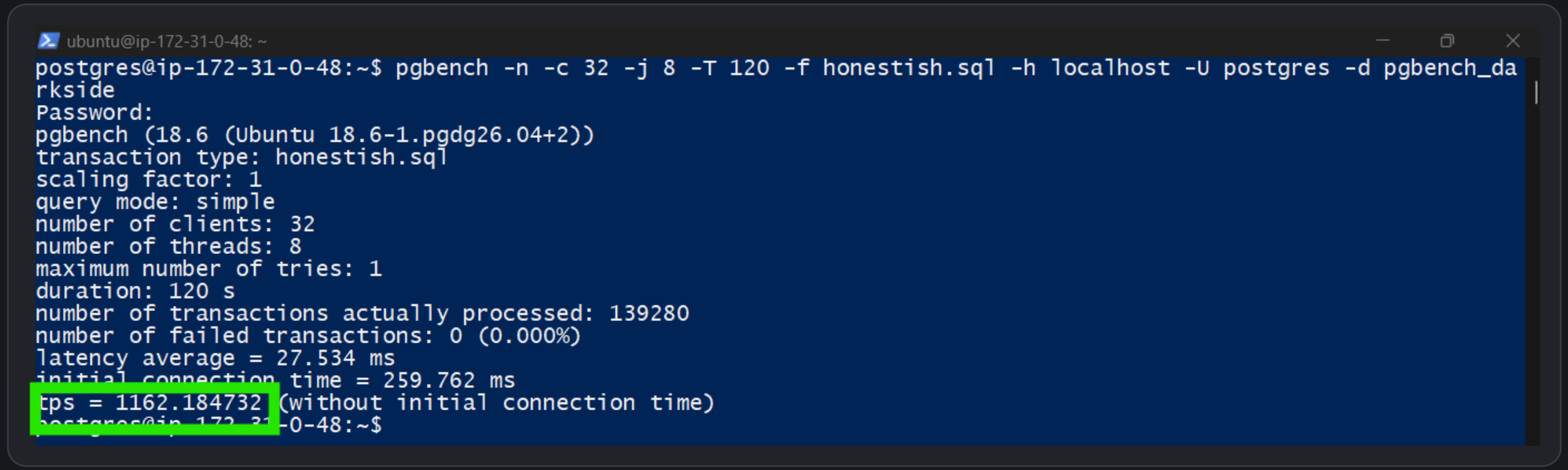
## MACHINE

| | |
|---|---|
| **Host** | Intel Core i7-13700, 16 cores / 24 threads |
| **RAM** | 63.8 GB |
| **Storage** | Samsung SSD 980 NVMe |
| **OS** | Windows 11 |
| **Database** | PostgreSQL 18.6, x86_64 |
| **Driver** | pgbench 18.6 |

## TEST SETUP

| | |
|---|---|
| **Test duration** | 120 seconds per measured test |
| **Load** | 32 clients / 8 threads |
| **Database** | `pgbench_darkside` |
| **Connection** | `-h localhost -U postgres -d pgbench_darkside` |
| **Password source** | `PGPASSWORD=*******` |
| **Dataset** | 100 branches, 1,000,000 accounts |

# Why not VM in a cloud?




**1,162 TPS**

cloud VM, 32 clients, 120 s

**27.5 ms**

average latency

# Baseline settings before the “cheats”

| | |
|---|---|
| jit | **off** |
| full_page_writes | **on** |
| shared_buffers | **1GB** |
| fsync | **on** |
| synchronous_commit | **on** |
| wal_compression | **LZ4** |

Windows machine • local NVMe • tuned OLTP-ish PostgreSQL configuration

# INTENTIONALLY FLAWED

## Case 0: sabotage the baseline

```
DELETE FROM bench_audit
WHERE audit_id IN (
  SELECT audit_id
  FROM bench_audit
  WHERE created_at < now() - interval '10 minutes'
  ORDER BY created_at
  LIMIT 1
);
```

NO INDEX ON CREATED_AT

**584**

TPS

**dishonest baseline**

This is not PostgreSQL being slow.
This is the schema being lazy.

# EXPLAIN ANALYZE

**WITHOUT INDEX**

```
===== PLAN WITHOUT composite index (ORDER BY created_at LIMIT 1) =====
                                QUERY PLAN
-----------------------------------------------------------------------
 Limit (actual rows=0.00 loops=1)
   Buffers: shared hit=1667
   ->  Sort (actual rows=0.00 loops=1)
         Sort Key: created_at
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=1667
         ->  Seq Scan on bench_audit (actual rows=0.00 loops=1)
               Filter: (created_at < (now() - '00:10:00'::interval))
               Rows Removed by Filter: 200000
               Buffers: shared hit=1667
 Planning:
   Buffers: shared hit=34
(12 rows)
```

**WITH COMPOSITE INDEX**

```
===== PLAN WITH composite index =====
                                                        QUERY PLAN
---------------------------------------------------------------------------------------------------------
 Limit (actual rows=0.00 loops=1)
   Buffers: shared hit=3
   -> Index Only Scan using bench_audit_created_at_audit_id_idx on bench_audit
         Index Cond: (created_at < (now() - '00:10:00'::interval))
         Heap Fetches: 0
         Index Searches: 1
         Buffers: shared hit=3
 Planning:
   Buffers: shared hit=9 read=4
(9 rows)
```

**Index Only Scan** reads 3 buffers, 0 heap fetches

# The plan tells you exactly what I broke

| WITHOUT INDEX | | WITH COMPOSITE INDEX | |
|---|---|---|---|
| 200,000 | rows removed by filter | 0 | heap fetches |
| 1,667 | shared buffers hit | 3 | shared buffers hit |
| Seq Scan | → Sort → Limit | Index Only | Scan → Limit |

HONEST BASELINE

# Add the obvious missing index.

```
CREATE INDEX bench_audit_created_at_audit_id_idx
ON bench_audit(created_at, audit_id);
```

**22,705**

TPS — HONEST BASELINE

41× OVER CASE 0

# FOUR LEVERS

1

**GUARANTEES**

durability / WAL / crash promises

2

**WORK**

do less or change persistence

3

**EXECUTION**

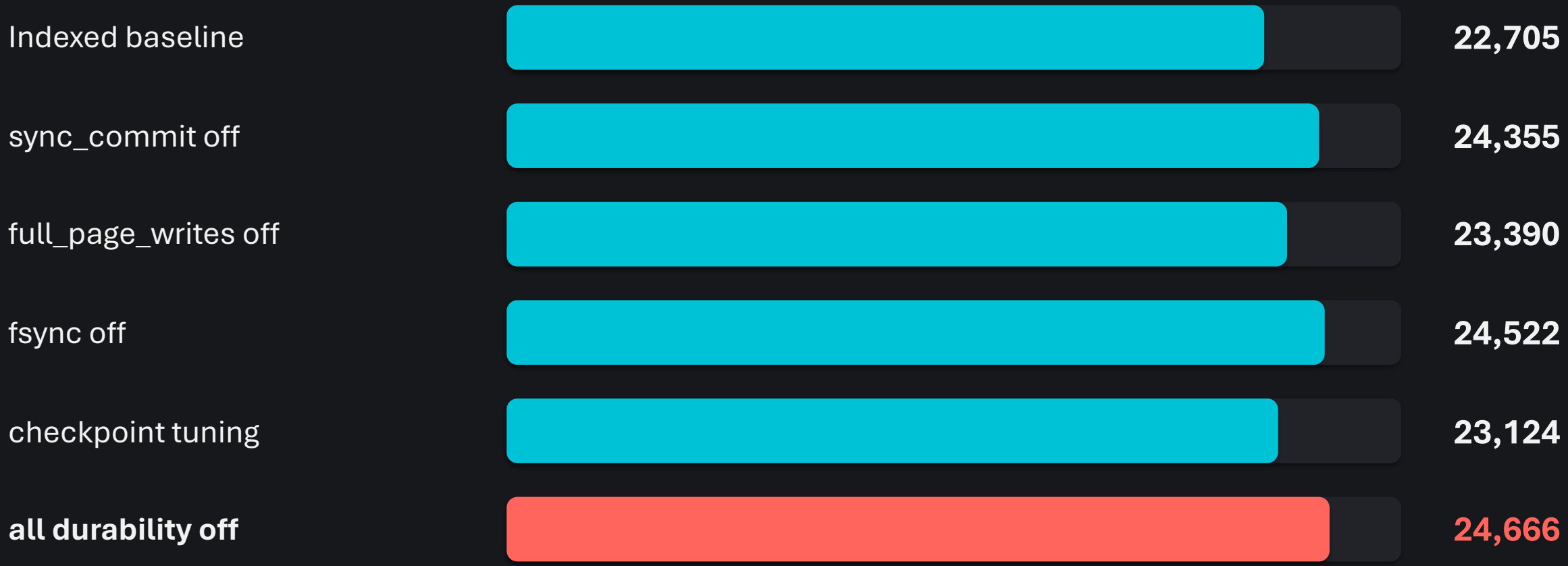
server-side logic / protocol

4

**METRIC**

batching and a new y-axis

# The scary durability knobs barely help



# Why Nuclear-sounding settings barely helped

| | default open_datasync | fdatasync |
|---|---|---|
| Measured latency | ~40 µs | ~2 ms |
| Write caching | Can remain enabled | Can prevent write caching |
| Benchmark impact | High TPS | Low TPS |

# UNLOGGED LEDGER / AUDIT

## Write less WAL

**LOGGED**

ledger
audit

→

**UNLOGGED**

ledger
audit

**37,780**

TPS

1.66×

# LESS WORK

- × SELECT account
- ✓ **UPDATE balance**
- ✓ **INSERT ledger**
- × FK check
- × INSERT audit
- × audit cleanup

**45,145**

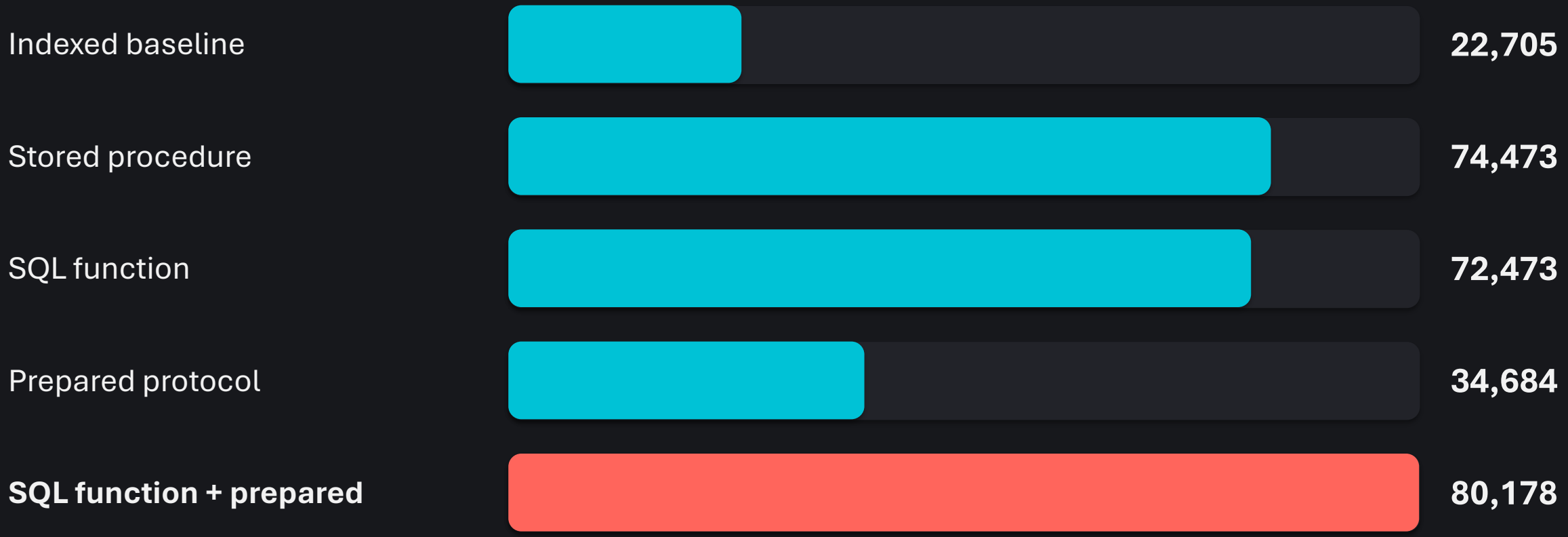
TPS

**1.99× VS INDEXED**

**Doing less work is faster.**

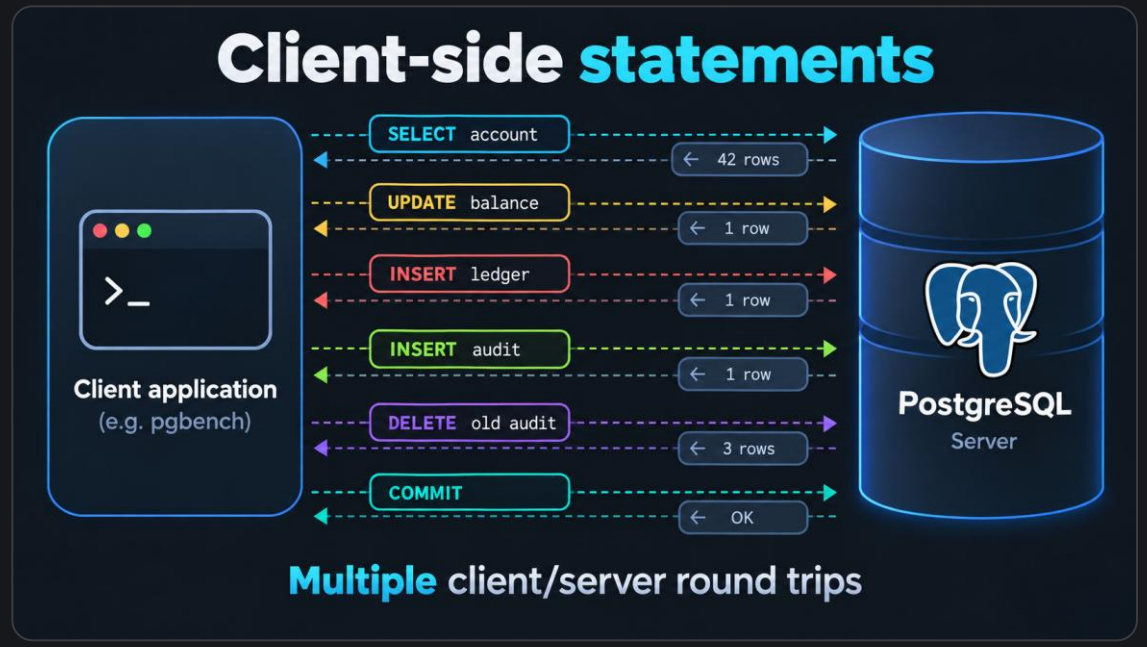
# SERVER-SIDE + PREPARED

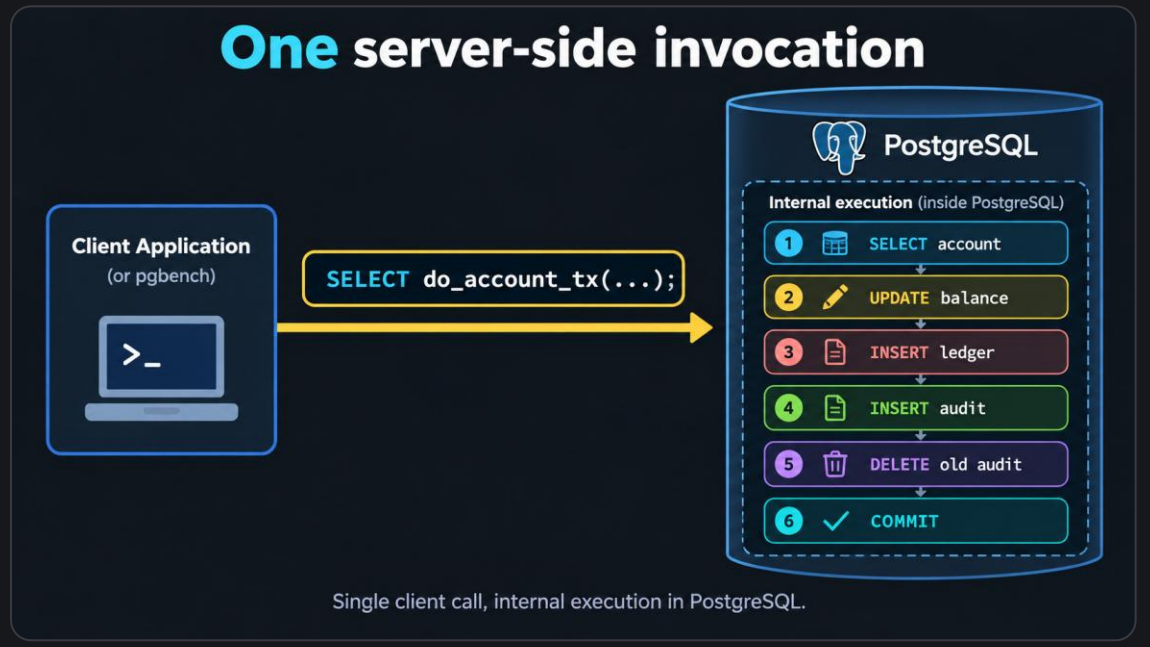
## Change the execution shape



# SERVER-SIDE FUNCTIONS

## Six round trips become one call




**BEFORE** Each statement waits on the network

**AFTER** The whole transaction runs inside PostgreSQL

# PREPARED PROTOCOL

## Parse and plan once, then only execute

### Simple protocol

Every statement, every time

Parse

↓

Analyze & rewrite

↓

Plan

↓

Execute

### Prepared protocol

After the first run, only Execute

Parse **skipped**

↓

Analyze & rewrite **skipped**

↓

Plan **skipped**

↓

**Execute**

**22,705 → 34,684 TPS (+53%).** Still six round trips per transaction; each statement just costs less CPU.

# The respectable hero result.

**80,178**

TPS

3.53× VS INDEXED

# This is where an honest benchmark pauses.

## THE PIVOT

# Up to here, the y-axis still means TPS.

Now the y-axis starts doing marketing work.

# CHANGE THE UNIT

## batch ×8

ONE TRANSACTION




8 account operations inside it

24,099

TPS

→

192,790

REPORTED OPS

8.49× HEADLINE

# BATCH ×32

## bigger batch, bigger headline

**8,426**
TPS

↓

**269,617**
REPORTED OPS

↑

Marketing metric rises.

11.87× HEADLINE

# BENCHMARKETING TRICK #2

**TPS ↓  OPS ↑  Headline ↑↑↑**

The number is real. The comparison changed.

# ALL CHEATS + BATCH ×32

## Grand finale: stack the deck

SQL FUNCTION

PREPARED PROTOCOL

32 OPS / TRANSACTION

RELAXED DURABILITY

LESS INTEGRITY WORK

**362,789**

REPORTED OPERATIONS / SECOND

# THE FULL SCOREBOARD

PostgreSQL 18.6 on Windows 11, trusted Dev Drive V: (ReFS), pgbench -c 32 -j 8 -T 120, 1 round

| Method | Protocol | TPS | OPS* | Latency (ms) | × baseline | × Case 0 |
|---|---|---|---|---|---|---|
| **Case 0: bad code / missing index** | simple | 584 | 584 | 54.80 | 0.03× | 1.0× |
| **Indexed baseline** | simple | 22,705 | 22,705 | 1.41 | 1.00× | 38.9× |
| **synchronous_commit=off** | simple | 24,355 | 24,355 | 1.31 | 1.07× | 41.7× |
| **full_page_writes=off** | simple | 23,390 | 23,390 | 1.37 | 1.03× | 40.1× |
| **fsync=off** | simple | 24,522 | 24,522 | 1.30 | 1.08× | 42.0× |
| **checkpoint tuning** | simple | 23,124 | 23,124 | 1.38 | 1.02× | 39.6× |
| **all durability off** | simple | 24,666 | 24,666 | 1.30 | 1.09× | 42.2× |
| **less work (drop FK/SELECT/audit)** | simple | 45,143 | 45,143 | 0.71 | 1.99× | 77.3× |
| **UNLOGGED ledger/audit** | simple | 37,780 | 37,780 | 0.85 | 1.66× | 64.7× |
| **stored procedure** | simple | 74,473 | 74,473 | 0.43 | 3.28× | 127.5× |
| **stored function (SQL)** | simple | 72,743 | 72,743 | 0.44 | 3.20× | 124.6× |
| **prepared protocol** | prepared | 34,684 | 34,684 | 0.92 | 1.53× | 59.4× |
| **sql function + prepared** | prepared | 80,178 | 80,178 | 0.40 | 3.53× | 137.3× |
| **batching ×8** | simple | 24,099 | 192,790 | 1.33 | 8.49× | 330.2× |
| **batching ×32** | simple | 8,426 | 269,617 | 3.80 | 11.87× | 461.7× |
| **all cheats + batch ×32 (SQL fn)** | **prepared** | **11,337** | **362,789** | **2.82** | **15.98×** | **621.3×** |

* OPS = operations/second for batched methods, TPS otherwise. × baseline = vs indexed baseline; × Case 0 = vs the crippled Case 0. Highlighted row stacks every cheat at once. Dev Drive with Defender performance mode keeps real-time scanning out of the commit path (scans run asynchronously).

# CHAMP IS HERE

**362,789**

REPORTED OPS

**15.98×**

VS INDEXED BASELINE

**621×**

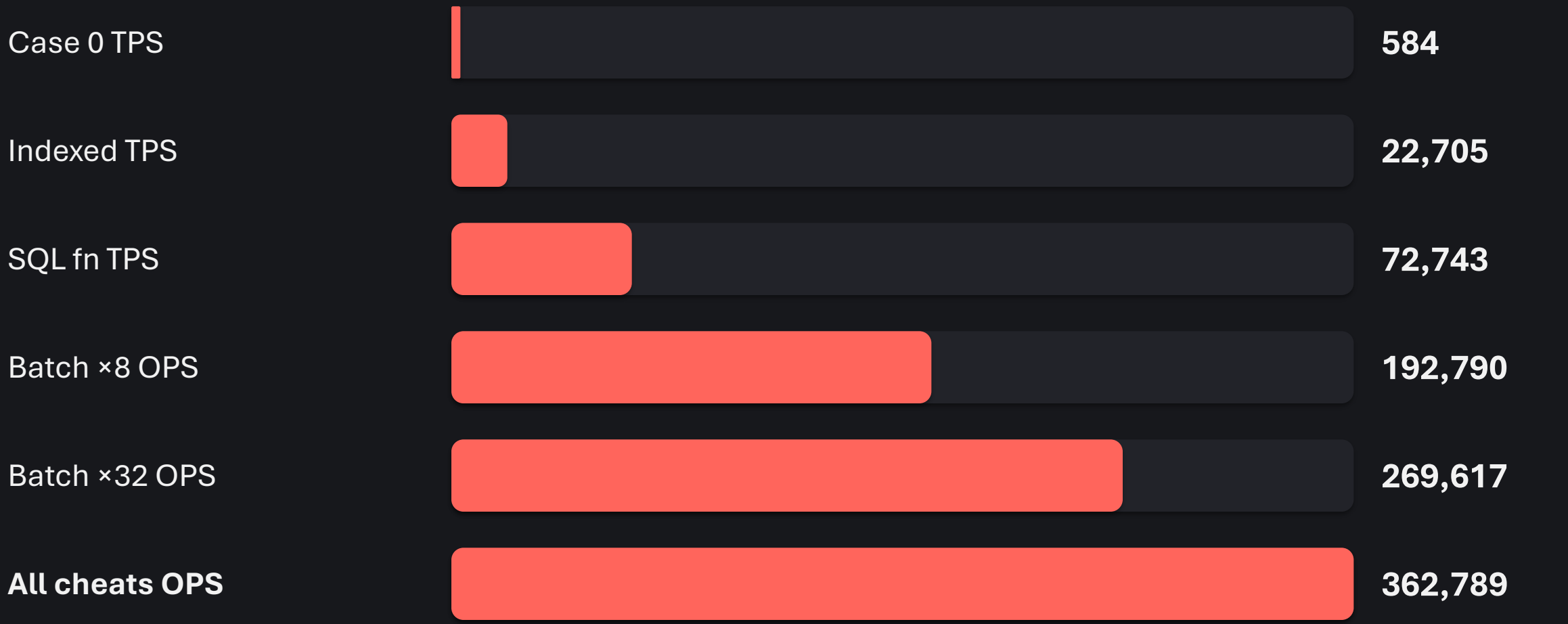
VS CRIPPLED CASE 0

The number is not fake. The framing is the trick.

# THE MARKETING CHART

## Now plot the headline number instead

**WARNING: TPS → OPS**



# Extended Trickery

## The trick: always benchmark against vanilla PostgreSQL.

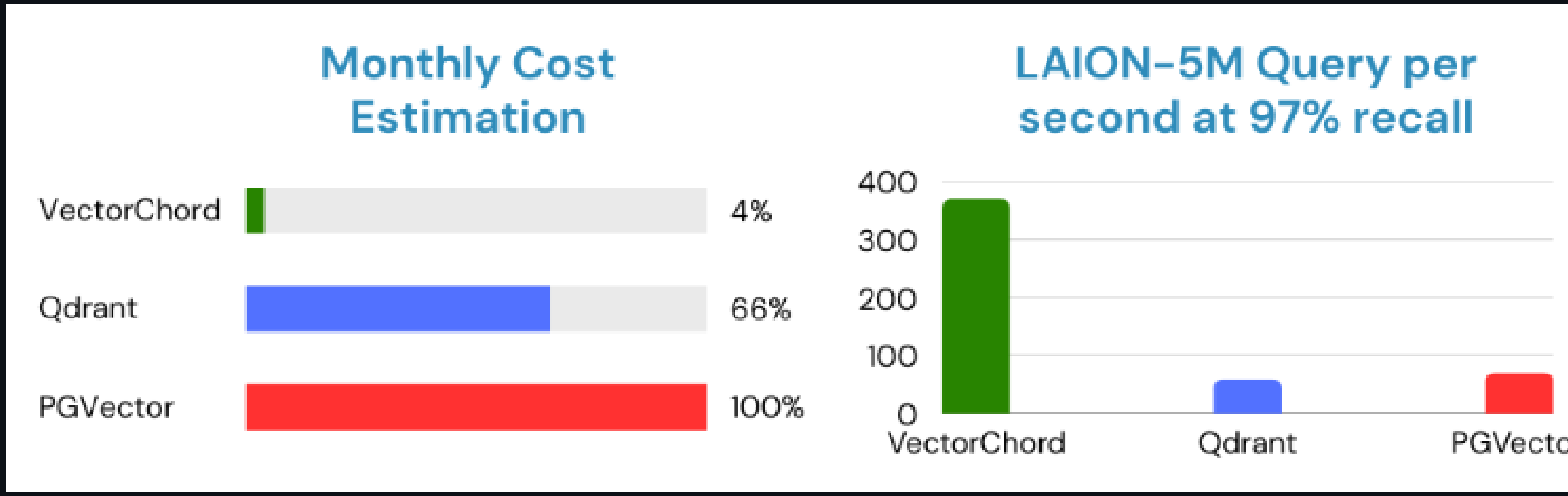
VECTOR SEARCH SPEED, RELATIVE TO PGVECTOR



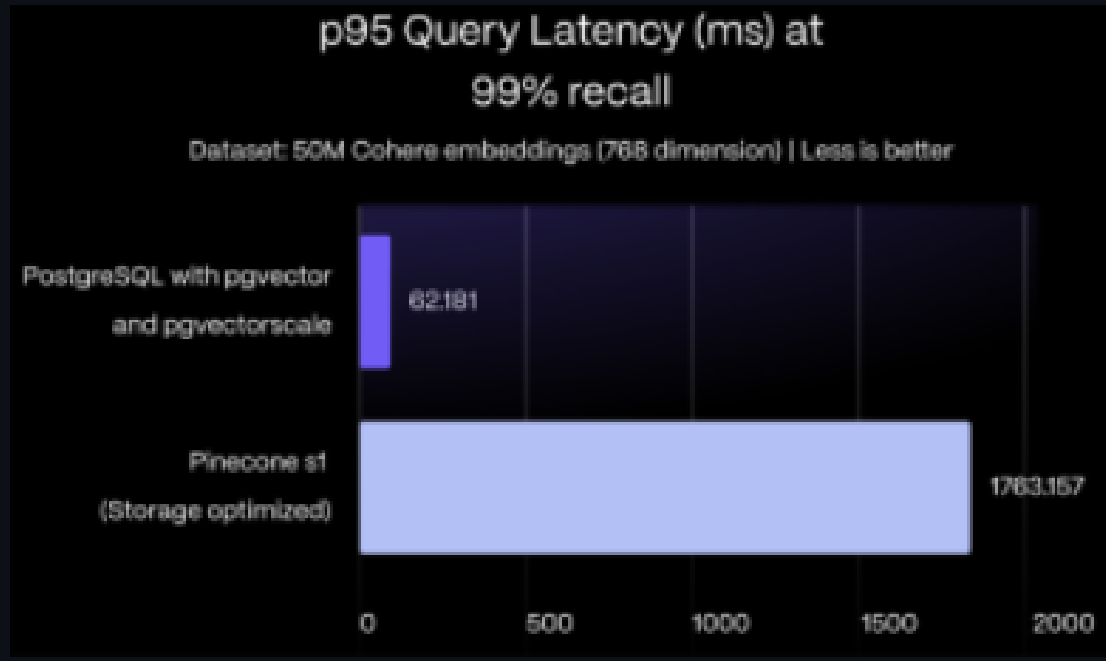

## B = 4A and C = 20A, so C = 5B?

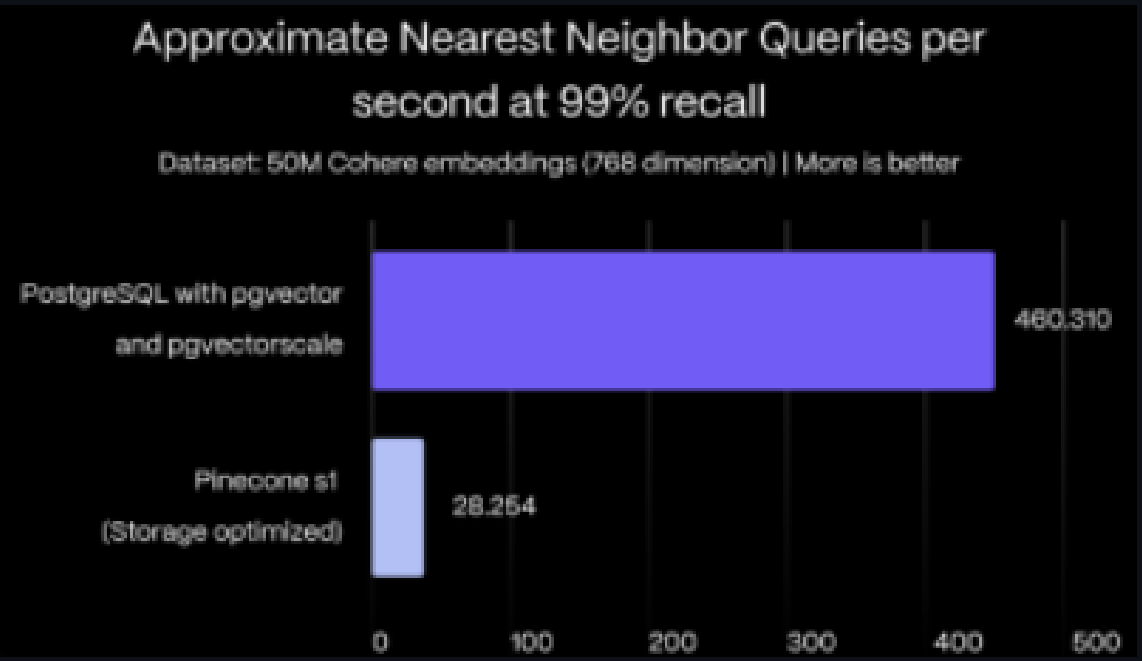
Against Postgres with the right extension, XYZdb is 5x slower.

400,000 vectors for just $1, enabling significant savings: 6x more vectors compared t
Pinecone's optimized storage and 26x more than pgvector/pgvecto.rs for the same p



On a benchmark dataset of 50 million Cohere embeddings with 768 dimensions each, PostgreSQL with `pgvector` and `pgvectorscale` achieves **28x lower p95 latency** and **16x higher query throughput** compared to Pinecone's storage optimized (s1) index for approximate nearest neighbor queries at 99% recall, all at 75% less cost when self-hosted on AWS EC2.



## THE HONEST CONCLUSION

# The number is not fake.

# The framing is the trick.

362,789 OPS. Same PostgreSQL. Different story.

# Before believing the next 100× graph...

Ask what changed between the bars.

# DECOMPOSE THE CLAIM

## Where did the big number really come from?

| | | |
|---|---|---|
| **Fix broken schema** | **584 → 22,705 TPS** | 39× headline opportunity |
| **Durability knobs** | **1.02×–1.09×** | mostly noise here |
| **Do less / UNLOGGED** | **1.99× / 1.66×** | work or guarantees changed |

# DECOMPOSE THE CLAIM

## The last jump comes from changing execution + metric

| | | |
|---|---|---|
| **Server-side SQL function** | **72,743 TPS** | 3.20× legitimate hero |
| **Batching** | **24,099 / 8,426 TPS** | headline switches to OPS |
| **Stack everything** | **362,789 OPS** | 15.98× vs indexed |

# BENCHMARK-READING CHECKLIST
# are we comparing the same thing?

| | | |
|---|---|---|
| 01 | **BASELINE** | Is the comparison system competently configured? |
| 02 | **WORK** | Are both sides doing the same logical work? |
| 03 | **GUARANTEES** | Same durability, consistency and crash semantics? |

# BENCHMARK-READING CHECKLIST

## Then: interrogate the metric and omissions

| | | |
|---|---|---|
| 04 | **METRIC** | TPS, queries, rows, requests… or “operations”? |
| 05 | **OMISSIONS** | Cost, p99 latency, write amplification, operational complexity? |
| 06 | **REPRODUCIBLE** | Can I see the code, config and workload? |

# PostgreSQL still gets one compliment

Even a dishonest benchmark had to use real PostgreSQL features to cheat convincingly.

- INDEXES
- BATCHING
- UNLOGGED TABLES
- STORED PROCEDURES
- SQL FUNCTIONS
- PREPARED STATEMENTS

**Mature. Boring. Production-grade machinery.**

# DATABASE WARS

## The details change. The crime scene stays familiar.

The same performance ladder could be built with MySQL, MariaDB, or almost any database.



# REFERENCES


01

**That time Oracle tried to have a professor fired for benchmarking their database**

Dan Luu

danluu.com/anon-benchmark/

02

**The DeWitt Clause: Why You Rarely See Database Benchmarks**

Brent Ozar

www.brentozar.com/archive/2018/05/the-dewitt-clause-why-you-rarely-see-database-benchmarks/

03

**The DeWitt clause censorship should be illegal**

David A. Wheeler

dwheeler.com/essays/dewitt-clause.html

THANK YOU

# Now go distrust some graphs.

## Questions?



Talk and scripts on GitHub

# See you at the Prague PostgreSQL Meetup




## RSVP

Scan to
grab a spot